

# Instance File Format

## 1. File name

Every benchmark instance follows this naming pattern:

```
MPVRP_A_sB_dC_pD.dat
```

| Field | Meaning                          |
|-------|----------------------------------|
| A     | Instance number, from 001 to 100 |
| B     | Number of service stations       |
| C     | Number of depots                 |
| D     | Number of products               |

For example:

```
MPVRP_001_s36_d8_p3.dat
```

Each file has a matching counterpart in both benchmark scenarios. The version with changeover costs is used for the official evaluation. The zero-cost version contains the same fleet, locations, stocks, demands, and identifier, but all values in its transition matrix are zero.

## 2. General structure

An instance is a plain-text file. Its sections always appear in this order:

```
# <uuid>
NbProducts NbDepots NbGarages NbStations NbVehicles
<product transition matrix>
<vehicles>
<depots>
<garages>
<service stations>
```

Values may be separated by spaces or tabs. Every numeric token after the UUID must be written as an integer; decimal notation such as 18.0 is rejected even when it represents a whole number. Identifiers begin at 1 and must remain consecutive within each category. Apart from the identifier on the first line, the file should not contain comments or blank lines.

### 3. Instance identifier

The first line contains the unique identifier shared by the two versions of an instance:

```
# c01ab718-9a2c-4a7d-bb95-f37e2a389409
```

### 4. Main dimensions

The second line gives the number of products, depots, garages, stations, and vehicles:

```
NbProducts NbDepots NbGarages NbStations NbVehicles
```

For example, the following line describes an instance with 3 products, 2 depots, 1 garage, 20 stations, and 5 vehicles:

```
3 2 1 20 5
```

### 5. Transition cost matrix

The next `NbProducts` lines form a square matrix:

```
Cost_P1_to_P1 Cost_P1_to_P2 ...
Cost_P2_to_P1 Cost_P2_to_P2 ...
...
```

The value on row `p` and column `q` is the operational cost of preparing the vehicle to load product `q` when its current configuration is product `p`. This cost may include the loading setup itself; it is not limited to cleaning or changing the product.

All values are non-negative integers. The matrix may be asymmetric because the preparation required from `p` to `q` can differ from the preparation required from `q` to `p`.

In cost-bearing instances, diagonal entries are positive integers from the `low` range `[25, 150]`: loading the same product again still represents a preparation and loading operation. Off-diagonal entries use the `normal` range `[1001, 3500]`, the `high` range `[4501, 15000]`, or a mixture of both ranges depending on the instance regime. In the paired zero-cost scenario, every matrix entry, including the diagonal, is replaced by `0`.

### 6. Vehicles

Each vehicle is described on one line:

```
ID Capacity HomeGarage InitialProduct
```

- `ID` identifies the vehicle.
- `Capacity` is the maximum integer quantity it can carry and must be positive.
- `HomeGarage` identifies the garage where its schedule starts and ends.
- `InitialProduct` gives its product configuration before the first loading.

The initial product is important because the first loading cost is read from the transition matrix using this configuration as the starting point.

## 7. Depots

Each depot is described as follows:

```
ID X Y Stock_P1 Stock_P2 ... Stock_Pn
```

`X` and `Y` are integer coordinates. The remaining values give the available integer stock of each product. Stocks must be non-negative, and the total stock of every product across all depots must cover total demand.

## 8. Garages

Each garage has an identifier and a position:

```
ID X Y
```

Coordinates are integers.

## 9. Service stations

Each station is described by:

```
ID X Y Demand_P1 Demand_P2 ... Demand_Pn
```

Coordinates and demand values are integers. Demands must be non-negative, and every station must request at least one product. Deliveries may be split between vehicles, but each station-product demand must be fully satisfied.

A vehicle may serve a station only once for the same product over its complete schedule. It may return to that station on another trip to deliver a different product. Consequently, when a station-product demand is split, each contributing share must be assigned to a different vehicle.

## 10. Complete example

```
# c01ab718-9a2c-4a7d-bb95-f37e2a389409
2 1 2 3 2
42 1018
1562 87
1 20000 1 1
2 20000 1 2
1 82 64 57914 82626
1 98 50
2 57 26
1 24 42 0 4278
2 4 38 1344 2350
3 57 31 0 2319
```

This instance contains 2 products, 1 depot, 2 garages, 3 stations, and 2 vehicles. Vehicle 1 initially carries product 1, while vehicle 2 initially carries product 2.

## 11. Distances derived by the software

Distances are not stored in the instance file. They are calculated from the integer coordinates using Euclidean distance rounded to the nearest integer:

$$distance = round(\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2})$$